

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Investigación: el patrón de diseño Builder

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

Boleta: 2019630177

id634296@gmail.com

7CM2

7 de septiembre de 2026

Índice

1. Introducción	2
2. Qué es el patrón Builder	2
3. Para qué sirve: el problema de los constructores telescópicos	2
4. Cómo se implementa	3
5. Las dos versiones, lado a lado	4
6. Dónde aparece en la biblioteca estándar	5
7. Variantes y ajustes	5
8. Cuándo no conviene usarlo	5
9. Conclusión	6
10. Referencias Bibliográficas	7

Índice de figuras

1. Los cuatro participantes del patrón Builder según el catálogo original 2

Índice de tablas

1. Diferencias entre la formulación original del patrón y la variante de uso común en Java 5

1. Introducción

El nombre «Builder» señala dos cosas distintas en la literatura de Java, y confundirlas es la fuente más común de malentendidos sobre este patrón. La primera es la definición original del catálogo de Gamma, Helm, Johnson y Vlissides [1], donde Builder es un patrón creacional que separa el proceso de construir un objeto complejo de la representación final que produce. La segunda es la recomendación de Joshua Bloch para clases con demasiados parámetros de constructor [2], que es la que aparece en casi todo el código Java que dice usar builders, aunque su estructura y su motivación no son las del catálogo.

Este documento cubre las dos. Empieza por la definición formal, sigue con el problema concreto que resuelve la variante de Bloch, muestra cómo se escribe una y otra, y termina con los casos donde la respuesta correcta es no usar el patrón. El ejemplo que recorre el texto es una clase **Carrera**, una entidad con pocos campos al principio y con varios opcionales después, que sirve para ver en qué momento el patrón empieza a valer la pena.

Queda fuera la generación automática de builders con bibliotecas externas como Lombok, más allá de una mención en la sección de variantes, porque depende de una herramienta que se instala aparte y no del lenguaje ni de su biblioteca estándar.

2. Qué es el patrón Builder

Gamma et al. lo clasifican entre los patrones creacionales, junto con Abstract Factory, Factory Method, Prototype y Singleton. El propósito, en su formulación original, es separar la construcción de un objeto complejo de su representación, de modo que el mismo proceso de construcción pueda producir representaciones distintas.

La estructura tiene cuatro participantes. El Builder declara una interfaz con un método por cada parte que compone el producto. Cada ConcreteBuilder implementa esa interfaz y arma su propia versión del resultado. El Director conoce el orden en que hay que llamar esos métodos, pero no sabe qué tipo concreto está saliendo del otro lado. El Product es el objeto terminado.

El ejemplo del libro es un conversor de documentos en formato RTF. El Director recorre el documento una sola vez, y según qué ConcreteBuilder reciba, ese mismo recorrido produce texto plano, TeX o un componente de interfaz gráfica. El proceso de lectura no cambia. Lo que cambia es quién responde a cada paso.

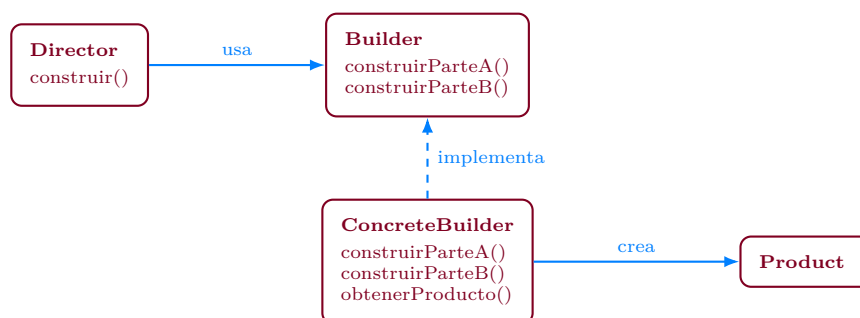


Figura 1: Los cuatro participantes del patrón Builder según el catálogo original

La figura 1 deja ver por qué el Director importa en esta versión: es el único que sabe la secuencia, y al depender solo de la interfaz Builder puede repetir esa secuencia con cualquier implementación. Cambiar el formato de salida no implica tocar el recorrido.

3. Para qué sirve: el problema de los constructores telescópicos

Una clase de tres atributos no necesita nada de esto. La clase **Carrera** con identificador, nombre y descripción se construye bien con un constructor normal, y agregarle un builder sería trabajo perdido. El problema empieza cuando la clase crece.

Supóngase que la misma entidad termina con clave, nombre, descripción, número de semestres, créditos totales, modalidad y un indicador de si sigue vigente. Solo dos de esos campos son obligatorios. Los demás tienen valores razonables por defecto o pueden faltar. La salida tradicional es escribir un constructor por cada combinación útil, cada uno llamando al siguiente:

Código 1: Constructores telescópicos: cada uno delega en el que tiene un parámetro más

```
1 public Carrera(String clave, String nombre) {
2     this(clave, nombre, "");
3 }
```

```

4
5 public Carrera(String clave, String nombre, String descripcion) {
6     this(clave, nombre, descripcion, 9);
7 }
8
9 public Carrera(String clave, String nombre, String descripcion, int semestres) {
10     this(clave, nombre, descripcion, semestres, 450, true);
11 }
12
13 public Carrera(String clave, String nombre, String descripcion,
14                 int semestres, int creditos, boolean vigente) {
15     this.clave = clave;
16     this.nombre = nombre;
17     this.descripcion = descripcion;
18     this.semestres = semestres;
19     this.creditos = creditos;
20     this.vigente = vigente;
21 }

```

Compila y funciona, pero tiene dos defectos que se agravan conforme se agregan campos. El primero es de lectura: en la llamada `new Carrera("ISC", "Ingeniería en Sistemas", "Plan 2020", 9, 450, true)` no hay forma de saber qué significan el 9, el 450 y el `true` sin abrir la definición de la clase. El segundo es más serio, porque el compilador no ayuda: dos parámetros consecutivos del mismo tipo se pueden intercambiar por accidente y el programa compila igual. Escribir `new Carrera(nombre, clave)` en vez de `new Carrera(clave, nombre)` produce un objeto con los valores cruzados y ningún aviso hasta que alguien lo note en tiempo de ejecución.

La otra salida conocida es el estilo JavaBean: un constructor vacío y un `set` por atributo. Se lee mucho mejor, porque cada valor queda etiquetado con el nombre del método. A cambio aparecen dos problemas nuevos. El objeto existe en un estado incompleto durante todo el tramo entre el constructor y la última llamada a un `set`, así que cualquier código que lo reciba a medio llenar lo va a tratar como si estuviera listo. Y como los `set` tienen que ser públicos para poder usarse desde fuera, la clase no puede ser inmutable. Bloch señala ambos costos al presentar la alternativa [2].

El builder da lo que tienen las dos opciones sin lo que les sobra: las llamadas quedan etiquetadas como en el estilo JavaBean, y el objeto aparece completo en un solo momento, el de la llamada final, con lo cual puede ser inmutable.

4. Cómo se implementa

La forma que describe Bloch usa una clase estática anidada dentro de la clase que se quiere construir. Los parámetros obligatorios entran por el constructor del builder, los opcionales por métodos que devuelven el propio builder, y un método final entrega el objeto terminado.

Código 2: La clase `Carrera` con su builder anidado

```

1 public class Carrera {
2
3     private final String clave;
4     private final String nombre;
5     private final String descripcion;
6     private final int semestres;
7     private final boolean vigente;
8
9     private Carrera(Builder builder) {
10         this.clave = builder.clave;
11         this.nombre = builder.nombre;
12         this.descripcion = builder.descripcion;
13         this.semestres = builder.semestres;
14         this.vigente = builder.vigente;
15     }
16
17     public static class Builder {
18
19         // obligatorios: van en el constructor del builder
20         private final String clave;
21         private final String nombre;

```

```

22
23 // opcionales: cada uno arranca con un valor por defecto
24 private String descripcion = "";
25 private int semestres = 9;
26 private boolean vigente = true;
27
28 public Builder(String clave, String nombre) {
29     this.clave = clave;
30     this.nombre = nombre;
31 }
32
33 public Builder descripcion(String descripcion) {
34     this.descripcion = descripcion;
35     return this;
36 }
37
38 public Builder semestres(int semestres) {
39     this.semestres = semestres;
40     return this;
41 }
42
43 public Builder vigente(boolean vigente) {
44     this.vigente = vigente;
45     return this;
46 }
47
48 public Carrera build() {
49     if (semestres <= 0) {
50         throw new IllegalArgumentException("Los semestres deben ser positivos");
51     }
52     return new Carrera(this);
53 }
54 }
55 }

```

Del lado de quien la usa, la construcción queda así:

Código 3: Uso del builder

```

1 Carrera isc = new Carrera.Builder("ISC", "Ingeniería en Sistemas Computacionales")
2     .semestres(9)
3     .descripcion("Plan de estudios 2020")
4     .build();

```

Cuatro decisiones de ese código son las que hacen que el patrón funcione, y conviene no perderlas de vista al escribirlo.

Cada método opcional termina con **return this**, y eso es todo lo que hace posible encadenar las llamadas. A ese estilo de encadenamiento se le llama interfaz fluida, y no es exclusivo del patrón, pero es lo que le da la legibilidad por la que se elige.

El constructor de **Carrera** es privado y recibe el builder. Como es el único constructor que existe, no hay manera de crear una **Carrera** sin pasar por **build()**. Eso convierte al builder en la única puerta de entrada.

Los campos del producto son **final**. Una vez que **build()** devuelve el objeto, nadie puede modificarlo, y por eso se puede compartir entre hilos o usar como llave de un mapa sin sobresaltos.

La validación vive en **build()**, en un solo lugar, y ahí puede revisar invariantes que involucren a varios campos a la vez, algo que un **set** suelto no alcanza a comprobar porque solo ve el valor que le llega.

5. Las dos versiones, lado a lado

Las dos cosas que se llaman Builder resuelven problemas diferentes con estructuras diferentes. La tabla 1 resume en qué se apartan.

	Catálogo original (1994)	Variante de Bloch
Problema que ataca	El mismo proceso debe producir representaciones distintas	La clase tiene demasiados parámetros de constructor
Director	Existe y coordina los pasos	No existe; el orden lo decide quien llama
Interfaz Builder	Sí, con varias implementaciones	No; una sola clase anidada
Tipo del producto	Cambia según el Concrete-Builder	Siempre el mismo
Resultado típico	Un documento o un árbol en varios formatos	Un objeto inmutable con muchos campos

Tabla 1: Diferencias entre la formulación original del patrón y la variante de uso común en Java

Vale la pena tener presente cuál se está usando al hablar del tema, porque muchos tutoriales presentan el código de Bloch bajo el diagrama del catálogo, con un Director que en ese código no aparece por ningún lado.

6. Dónde aparece en la biblioteca estándar

El patrón no es una construcción de manual sin uso real. La propia biblioteca de Java lo aplica en varios lugares, y reconocerlos ayuda a ubicar cuándo tiene sentido escribir uno propio.

`HttpRequest.newBuilder()`, del paquete `java.net.http` disponible desde Java 11, es la variante de Bloch casi textual: se encadenan la URI, el método, los encabezados y el tiempo de espera, y `build()` devuelve un `HttpRequest` inmutable [3]. `DateTimeFormatterBuilder` se acerca más al catálogo original, porque se le van agregando piezas de formato una por una y al final `toFormatter()` entrega el objeto armado. `Locale.Builder`, `Calendar.Builder` y `Stream.builder()` siguen la misma idea.

`StringBuilder` merece un comentario aparte, porque se cita como ejemplo del patrón con más frecuencia de la que le corresponde. Comparte el encadenamiento de `append` y tiene un paso final que produce el resultado, que es `toString()`. No tiene Director, ni interfaz con varias implementaciones, ni produce representaciones alternativas. Es un acumulador mutable con interfaz fluida, que se parece al patrón en la forma de escribirlo pero no en su estructura.

7. Variantes y ajustes

Un builder plano no obliga a nada en tiempo de compilación: quien lo use puede llamar `build()` antes de tiempo y enterarse del faltante hasta que se lance la excepción. Meter los campos obligatorios en el constructor del builder, como en el ejemplo de arriba, cubre buena parte del problema. Para los casos donde además importa el orden, existe el builder por etapas, que devuelve una interfaz distinta en cada paso, de manera que `build()` solo aparece como opción cuando ya se llenó todo lo necesario. Cuesta más clases y se justifica cuando el objeto es delicado de construir.

Para jerarquías de clases, Bloch describe un builder abstracto con genéricos recursivos, del estilo `Builder<T extends Builder<T>>` más un método `self()`, para que los builders de las subclases puedan encadenar los métodos heredados sin perder el tipo concreto [2]. Sin ese arreglo, encadenar un método del padre devuelve el tipo del padre y rompe el resto de la cadena.

Otro ajuste frecuente en objetos inmutables es un método `toBuilder()`, que devuelve un builder ya cargado con los valores actuales del objeto. Sirve para producir copias modificadas sin escribir todos los campos otra vez.

La anotación `@Builder` de Lombok genera todo este código en tiempo de compilación a partir de la clase. Ahorra la escritura, a cambio de una dependencia externa y de que el código que realmente se ejecuta no esté a la vista en el archivo fuente.

8. Cuándo no conviene usarlo

El costo del patrón es visible: cada campo se declara dos veces, una en el producto y otra en el builder, y encima lleva un método propio. Una clase de dos o tres parámetros queda más clara con un constructor normal o con un método estático de fábrica. Bloch ubica el punto de equilibrio alrededor de cuatro parámetros, y advierte que conviene decidirlo al diseñar la clase, porque migrar de constructores a builder después implica tocar todas las llamadas existentes [2].

Cada construcción crea además un objeto intermedio que se descarta enseguida. En la mayoría de los programas eso es irrelevante, pero en código muy sensible al rendimiento es un dato a considerar.

Hay un caso más donde el patrón aporta poco: cuando el objeto va a ser mutable de todos modos. Buena parte del beneficio viene de poder cerrar el objeto al terminar de construirlo, y si de cualquier forma va a exponer métodos `set`, el builder agrega código sin resolver el problema que justifica su existencia.

9. Conclusión

Builder resuelve dos problemas emparentados pero distintos. En su forma original aísla el proceso de construcción de la representación que produce, y ahí el Director es la pieza central. En la forma que se usa a diario en Java, sustituye un constructor con demasiados parámetros por una secuencia de llamadas con nombre, y su ganancia real es que el objeto nace completo y puede quedar inmutable.

La decisión de usarlo depende de una cuenta bastante concreta: cuántos parámetros tiene la clase, cuántos de ellos son opcionales, y si el objeto debe poder cambiar después de creado. Con pocos campos y un objeto mutable, el patrón agrega trabajo sin devolver nada. Con muchos campos opcionales y un objeto que conviene congelar, ahorra los errores silenciosos que produce una lista larga de argumentos posicionales.

Angel Frausto Robles

10. Referencias Bibliográficas

- [1] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN: 978-0201633610.
- [2] Joshua Bloch. *Effective Java*. 3.^a ed. Addison-Wesley, 2018. ISBN: 978-0134685991.
- [3] Oracle Corporation. *Java Platform SE API Specification: HttpRequest.Builder*. URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.net.http/java/net/http/HttpRequest.Builder.html> (visitado 07-09-2026).